

PSet #2 — Pipeline de datos end-to-end: demanda de emergencias ECU 911

Jarod Tierra · Felipe Rodríguez · Santiago Rodríguez · Juan Diego Cadena · Miguel Jarrín

Ciencia de Datos: end-to-end · Repositorio: github.com/junilai/Pset_2 · Octubre 2026

Contexto. En el PSet #1 definimos un modelo que estima la probabilidad de que un cantón tenga un día de demanda inusualmente alta de emergencias en t+3 y t+7 (volumen > P90 histórico de ese cantón y día de la semana). Este pipeline entrega la tabla que ese modelo necesita: **1 fila por cantón y día**, con conteos, calendario y features temporales. El P90 y el target no se calculan aquí: se calculan en la etapa de ML, solo con el periodo de entrenamiento, para evitar leakage.

1. Arquitectura

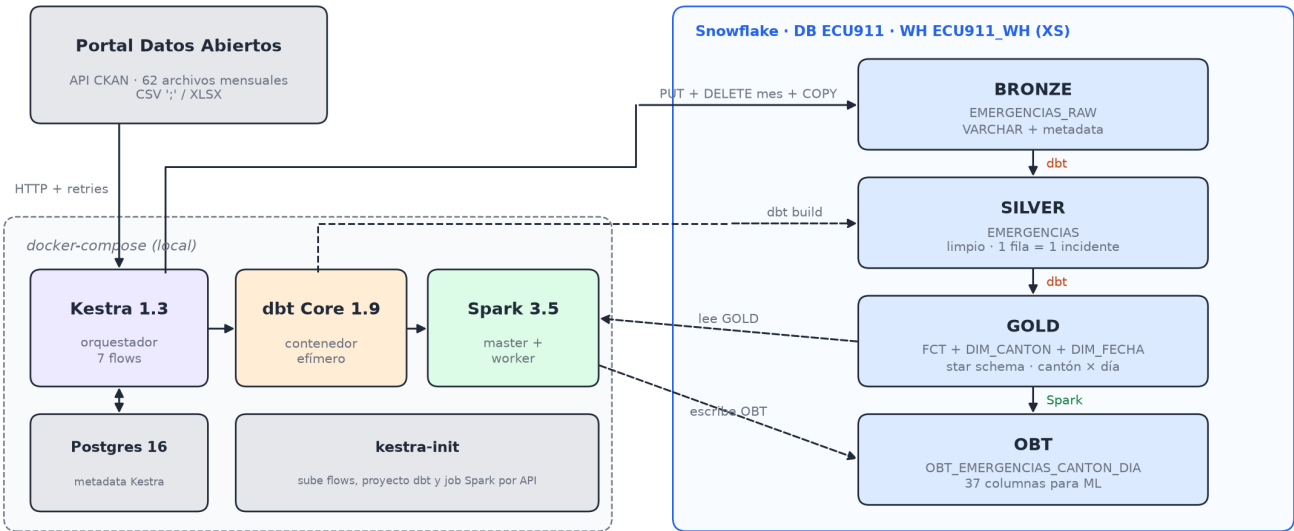


Figura 1. Infraestructura implementada. Flechas sólidas: carga de datos; punteadas: transformación.

Todo lo local se levanta con `docker compose up -d`: **Kestra 1.3 LTS** (con Postgres 16 solo para su metadata: flows, ejecuciones, estado de triggers), **Spark 3.5.9** standalone (master + worker, 2 cores/2 GB) y una imagen propia **dbt Core 1.9 + dbt-snowflake** construida desde `docker/dbt/Dockerfile` (nativa ARM64/AMD64). El servicio `kestra-init` espera a que Kestra esté listo y sube por API los 7 flows, el proyecto dbt y el job de Spark a los *namespace files*; así el código vive solo en Git y Kestra siempre ejecuta la versión del repositorio. Kestra lanza dbt y Spark como **contenedores efímeros** (montando `docker.sock`). Snowflake es el único destino de datos: database ECU911 con schemas BRONZE, SILVER, GOLD y OBT, warehouse X-Small con auto-suspend de 60 s, y un rol y usuario de servicio creados por el flow `snowflake_setup`. Las credenciales viajan `.env` → variables de entorno → `{{ envs.* }}` / `env_var()`; ninguna está en Git.

Cadena orquestada: Schedule (día 10) → `ingest_scheduled` → `ingest_month` ×N → `transform` → `dbt_build` → `spark_obt` → `reconciliar`. `transform` usa subflows con `wait` y `transmitFailed`: si un test de dbt falla, Spark no corre y la OBT conserva su última versión válida. El paso final consulta las 4 capas y falla si los conteos no cuadran. Para reproducir desde cero existe `run_all` (setup + ingesta por rango de meses + transform en una ejecución).

2. Ingesta con Kestra

Aspecto	Implementación
Fuente	ECU 911 «Base de Emergencias», portal datosabiertos.gob.ec (API CKAN). 1 archivo por mes, 2021-07 → 2026-08 (62 archivos), 7 columnas: Fecha, provincia, Canton, Cod_Parroquia, Parroquia, Servicio, Subtipo. ~270–310 mil filas/mes, 17,680,253 en total. No existe ID de incidente.
Granularidad	Fuente: 1 fila = 1 incidente (fecha sin hora). Unidad de carga: 1 mes (= 1 archivo publicado).
Trigger y frecuencia	Schedule cron <code>0 6 10 * * *</code> (día 10, 06:00 America/Guayaquil), <code>recoverMissedSchedules: ALL</code> . Recarga los 2 meses anteriores : el portal publica con 1–8 semanas de retraso y sin día fijo, y a veces re-publica meses.
Proceso de carga	catálogo CKAN → resolver URL del mes (jq) → falla con mensaje claro si aún no está publicado → descarga → si es XLSX se convierte a CSV (2026-04/05 llegaron en XLSX; la extensión del catálogo no es confiable) → PUT a un stage interno en carpeta <code>mes/execution_id</code> → BEGIN; DELETE mes; COPY INTO ... FORCE=TRUE; COMMIT → conteo → falla si 0 filas → limpieza del stage.
Idempotencia	Reemplazo de partición por mes en una transacción. Sin ID no se puede hacer MERGE; borrar el mes y recargarlo garantiza que re-ejecutar no duplica (verificado: 2026-08 recargado 2 veces = 280,867 filas, 1 ejecución por mes).

Aspecto	Implementación
Retries	HTTP (catálogo y descarga): exponencial 30 s → 5 min, 4 intentos. Snowflake: constante 30 s, 3 intentos. Se observó un fallo real del portal (conexión rechazada) que se recuperó en el reintento. Concurrencia máx. 4 cargas (QUEUE). Los subflows usan allowFailure: un mes no publicado no bloquea los demás.
Backfill	Backfill nativo de Kestra sobre el trigger monthly (2021-08-01 → hoy) con inputs meses = 1 y transformar = false para no correr dbt+Spark 62 veces; luego un solo transform. Resultado: 62/62 SUCCESS en ~25 min, y el trigger vuelve solo a su calendario.
Bronze	BRONZE. EMERGENCIAS_RAW: las 7 columnas como VARCHAR (dato original intacto, sin conversión) + metadata _PERIODO, _URL_ORIGEN, _ARCHIVO_STAGE, _FILE_ARCHIVO, _CARGADO_EN, _EJECUCION_KESTRA.

3. Calidad de datos

El perfilado se hizo sobre Bronze con SQL (docs/data_quality.sql) antes de escribir cualquier limpieza, revisando completitud, precisión, consistencia y validez. La regla general fue **marcar en vez de borrar**: Silver conserva exactamente las 17,680,253 filas de Bronze (lo verifica un test) y las exclusiones se hacen de forma explícita en Gold.

Dimensión / problema	Evidencia	Acción (dbt)	Justificación
Completitud: filas sin ubicación	2,127 vacías + 17 'NULL' + 16 '0' = 2,160 (0.012%)	Normalizar a NULL; bandera es_ubicacion_valida = false; no cuentan en Gold	No se pueden asignar a un cantón; marcar permite auditar sin perder la fila
Validez: Cod_Parroquia sin cero inicial	1,265,543 filas (7.2%) en 10 meses: 90150 vs 070150	LPAD(cod, 6, '0')	El código DPA del INEC tiene 6 dígitos; tras rellenar, el prefijo coincide con la provincia
Precisión: código por defecto 090150 en Morona Santiago	44 filas (0.02% de la provincia) en 9 cantones	Código de cantón = código más frecuente por (provincia, cantón)	El nombre es confiable; el código es un valor por defecto del sistema
Consistencia: cantones homónimos	BOLÍVAR (Carchi/Manabí), OLMEDO (Loja/Manabí): 223 nombres vs 224 cantones	Clave = código DPA de 4 dígitos, nunca el nombre	Agrupar por nombre sumaría cantones distintos
Consistencia: codificación	LOGROÑO en 4,637 filas (58 meses) vs LOGROÑO en 234	REPLACE('Đ','Ñ') y TRIM en todos los textos	Mismo código 1410: es un error de encoding de la fuente
Validez: dos formatos de fecha	Serial de Excel en 559,737 filas; d/m/yyyy con y sin cero inicial	COALESCE(TRY_TO_DATE(d/m/yy yy), serial + 1899-12-30)	100% de fechas interpretadas y 0 fuera de su mes (test)
Duplicados exactos	11,747,431 filas (66.4%) repiten fecha + lugar + servicio + subtipo	No eliminar ; incidente_id = periodo + fila del archivo	Sin hora ni ID, dos incidentes iguales el mismo día son legítimos; deduplicar destruiría el target
Consistencia temporal: 2024-01	~28.2% de volumen/día vs mediana de vecinos; 20/31 días bajos; todas las provincias	Conservar y marcar es_periodo_anomalo	Causa no confirmada (¿archivo incompleto? ¿estado de excepción?); la exclusión se decide en ML
Consistencia geográfica: Sevilla Don Bosco	Parroquia de Morona hasta 2025-02 (11,018 filas); cantón 1413 desde 2025-01	Reasignar filas antiguas al cantón 1413 (seed cantones_reasignados)	Evita un quiebre artificial de ~12% en Morona y ceros falsos en 1413
Completitud: cantón-días sin filas	26,716 de 422,912 cantón-días (6.32%) no aparecen en la fuente	Date spine con 0 en Gold	Un día sin emergencias es un 0 real; sin él, lags y P90 se sesgan hacia arriba

4. Transformaciones y modelado (dbt)

Bronze → **Silver**. SILVER.EMERGENCIAS lee con source('bronze', 'emergencias_raw') (con freshness: warn 45 días, error 75 días) y aplica todas las decisiones de la tabla anterior mediante macros reutilizables (limpiar_texto, es_periodo_anomalo). Grain: 1 fila = 1 incidente. **Silver** → **Gold**. Los modelos Gold solo usan ref(). dbt resuelve el orden de ejecución: seeds → Silver → dims → hecho. Materialización table (reconstrucción completa en ~45 s en un warehouse XS; incremental no compensa la complejidad hoy).

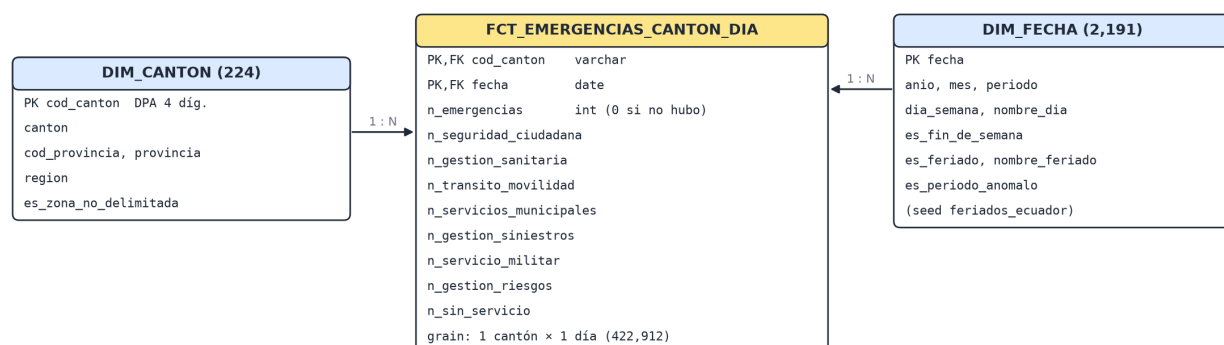


Figura 2. Star schema de la capa GOLD.

Grain del hecho FCT_EMERGENCIAS_CANTON_DIA: **1 cantón × 1 día calendario**, incluidos los días con 0 emergencias. Es el grain del target del PSet #1. El spine es dim_canton × dim_fecha, limitado al rango con datos para no inventar ceros en meses aún no publicados: 224 cantones × 1,888 días = **422,912 filas**, y la suma da 17,678,093 (= Silver válidas). Decisiones: los 7 servicios van como **columnas** del hecho, no como dim_servicio, porque son un conjunto fijo y multiplicarían el hecho ×8 sin cambiar el target. Las claves son naturales (código DPA y DATE): no hay historia que versionar y simplifican los joins en Spark. Los feriados nacionales (83 días 2021–2026, con traslados y decretos) son un seed versionado.

Tests (39, todos PASS). Además de los genéricos, los singulares codifican reglas reales de los datos:

Test	Regla que protege
unique + not_null incidente_id; silver_conserva_filas_de_bronze	un mes cargado dos veces; la limpieza marca, no borra
silver_fecha_dentro_de_su_mes; silver_meses_completos; silver_sin_meses_faltantes	confusión día/mes; archivo con días faltantes; mes ausente en el rango (generaría ~30 días de ceros falsos)
silver_volumen_mensual_estable (±20% vs mediana ±3 meses); silver_pct_sin_ubicacion_bajo (≤0.1%)	archivo truncado o duplicado; degradación de la calidad de ubicación
unique/not_null en PK; accepted_values región y día; relationships hecho → dims	integridad del star schema
fct_grain_canton_dia_unico; fct_spine_completo; fct_conserva_emergencias_de_silver; fct_servicios_suman_total; fct_canton_con_datos_cada_mes	grain; ceros completos; la agregación no pierde ni inventa; servicio nuevo sin columna; cambio de límites

Prueba de que los tests detectan problemas reales: con `--vars "{periodos_anomalos: []}"` falla silver_volumen_mensual_estable en 2024-01 y dbt salta Gold; Spark tampoco corre.

5. Spark y OBT

El job `spark/jobs/build_obt.py` (PySpark 3.5, conector `spark-snowflake` 3.2.2) lee el hecho y las dos dimensiones de GOLD, hace **LEFT JOIN con broadcast** de las dimensiones (224 y 2,191 filas: se copian a cada executor y se evita el shuffle) y calcula features con `Window.partitionBy(cod_canton).orderBy(fecha)`. El resultado se escribe en OBT.OBT_EMERGENCIAS_CANTON_DIA en modo `overwrite`, por lo que el job es idempotente.

Grain de la OBT: 1 cantón × 1 día, el mismo del hecho. Tiene 37 columnas: clave (2), cantón (5), calendario (11), medidas (9), features `lag_1d/7d/14d/28d` y `media_prev_7d/28d`, que usan solo días anteriores a `t` y quedan NULL si la ventana está incompleta; `es_feriado_t3/t7`, porque el calendario futuro se conoce de antemano; y los insumos del target `obj_n_emergencias_t3/t7`. El prefijo `obj_` indica que no deben usarse como features. Los lags se calculan por posición de fila y son correctos porque el spine no tiene huecos, lo que garantiza un test de dbt.

Validación de joins. Antes de escribir, el job verifica 7 condiciones y termina con error sin escribir si alguna falla:

Validación	Esperado	Obtenido
Filas OBT = filas hecho (sin multiplicación por joins)	422,912	422,912
Claves (cod_canton, fecha) distintas = filas (sin duplicados)	422,912	422,912
Σ n_emergencias OBT = Σ hecho	17,678,093	17,678,093
Filas sin match en dim_canton / dim_fecha	0 / 0	0 / 0
Nulos en es_feriado_t3 / es_feriado_t7	0 / 0	0 / 0

Al terminar, Kestra reconcilia las capas: **BRONZE 17,680,253 = SILVER 17,680,253 filas | SILVER válidas 17,678,093 = GOLD = OBT emergencias | GOLD 422,912 = OBT 422,912 cantón-días**. También se verificaron casos puntuales. Para Quito, el 22-dic-2025 tiene `media_prev_7d` = 2,145.14, que es el promedio del 15 al 21 de diciembre, y `es_feriado_t3` = true por el 25 de diciembre.

Star schema vs OBT. El star schema en GOLD es la fuente de verdad para análisis y BI: las dimensiones se reutilizan, no hay redundancia y es fácil agregar hechos, por ejemplo población INEC. La OBT es una vista derivada para el modelo: cada fila es una observación con todas sus features y no requiere joins al entrenar. Si cambia una dimensión, se actualiza GOLD y se regenera la OBT.

6. Batch vs. streaming

Batch es suficiente. La fuente se publica **una vez al mes**, con 1 a 8 semanas de retraso, como archivo completo. No hay eventos continuos que consumir. El modelo predice `t+3` y `t+7` para planificar turnos y recursos semanales, por lo que una latencia de días es aceptable. El pipeline completo tarda ~7 min y cuesta minutos de un warehouse XS al mes. Streaming agregaría infraestructura permanente, como Kafka y compute siempre encendido, sin mejorar la decisión.

Cuándo usaríamos streaming: si el ECU 911 expusiera un feed de incidentes en tiempo real (API o CDC del sistema de despacho) y el caso de uso cambiara a decisiones intradía, por ejemplo reubicar ambulancias según los picos de las próximas horas o alertar en minutos. Entonces el grain pasaría a cantón × hora, la ingesta sería un consumidor continuo

(Kafka + Snowpipe Streaming / Spark Structured Streaming) y las features se calcularían en ventanas deslizantes con manejo de eventos tardíos.

7. Limitaciones

- **Correcciones de meses antiguos:** el schedule solo recarga los 2 últimos meses. Si el portal re-publica un mes viejo, no se detecta automáticamente (el servidor expone ETag/Last-Modified, pero no se usan). Hoy se resuelve con un backfill manual.
- **Enero 2024** queda marcado como anómalo, pero no se sabe si el archivo está incompleto o si refleja una caída real.
- **Duplicados exactos:** sin hora ni ID no es posible distinguir un incidente real repetido de un registro duplicado por error.
- **Geografía vigente sin historia (sin SCD):** la reasignación de Sevilla Don Bosco se maneja con un seed manual. Si se crean nuevos cantones, habrá que actualizar ese seed. Un test lo detecta.
- **Faltan variables externas** mencionadas en el PSet #1: población INEC para calcular tasas y clima. Los feriados son un seed que hay que actualizar cada año; si dim_fecha no cubre t+7, el job de Spark falla con un mensaje explícito.
- **Reconstrucción completa** de Silver, Gold y OBT en cada corrida. Es viable con 17.7 M filas, pero si el volumen crece convendría usar modelos incrementales por periodo.
- **Infraestructura local** con un solo worker de Spark, sin alta disponibilidad. Kestra depende del host que ejecuta Docker.

Anexo A. Reproducción del pipeline

- `cp .env.example .env` y completar la cuenta y el usuario ACCOUNTADMIN de Snowflake. Luego ejecutar `docker compose up -d`; Kestra queda en localhost:8080 y Spark UI en localhost:8090.
- Kestra → `ecu911.run_all` → Execute con desde = 2021-07, hasta = 2026-08 y `setup = true`. Ejecuta setup, carga los 62 meses con 4 en paralelo y luego dbt, Spark y la reconciliación.
- Por partes: `snowflake_setup` → backfill del trigger de `ingest_scheduled` (`meses=1`, `transformar=false`) → `transform`.
- dbt manual: `docker compose run --rm dbt build (PASS=39)`. Spark manual: `spark-submit ... /opt/spark-jobs/build_obt.py`; ver README.

Anexo B. Conteos de referencia (histórico 2021-07 → 2026-08)

Objeto	Filas	Nota
BRONZE.EMERGENCIAS_RAW	17,680,253	62 meses, 1 ejecución por mes
SILVER.EMERGENCIAS	17,680,253	2,160 sin ubicación; 208,194 marcadas como periodo anómalo
GOLD.DIM_CANTON / DIM_FECHA	224 / 2,191	fechas 2021-01-01 → 2026-12-31
GOLD.FCT_EMERGENCIAS_CANTON_DIA	422,912	Σ n_emergencias = 17,678,093; 6.32% cantón-días en 0
OBT.OBT_EMERGENCIAS_CANTON_DIA	422,912	37 columnas; nulos de lag_28d = 6,272 (224 × 28)

Anexo C. Flows de Kestra

Flow	Función
<code>snowflake_setup</code>	Crea warehouse, database, schemas, rol y usuario (idempotente, IF NOT EXISTS)
<code>ingest_month</code>	Carga un mes a BRONZE (input month YYYY-MM); retries HTTP y Snowflake
<code>ingest_scheduled</code>	Trigger mensual + backfill; carga N meses y lanza transform
<code>transform</code>	<code>dbt_build</code> → <code>spark_obt</code> → reconciliación de capas
<code>dbt_build / spark_obt</code>	Corren dbt (freshness + build) y el job de Spark en contenedores efímeros
<code>run_all</code>	Setup + ingesta de un rango de meses + transform en un clic